

Πανεπιστήμιο Πειραιώς

Τμήμα Ψηφιακών Συστημάτων

Π.Μ.Σ «Πληροφοριακά Συστήματα και Υπηρεσίες»

Κατεύθυνση: «Μεγάλα Δεδομένα και Αναλυτική»

Α’ εξάμηνο

Μάθημα: Προγραμματισμός και Υποδομές Μεγάλων Δεδομένων: Python και Νεφοϋπολογιστική

Μέρος Β΄: Cloud

Καθηγητής: Κουσιουρής Γεώργιος

Φοιτητές: Παπακωνσταντίνου Αντώνιος, Κωστήρης Δημήτριος

ΑΜ: ΜΕ25028, ΜΕ25021

ΜΕΤΑΠΤΥΧΙΑΚΗ ΑΠΑΛΛΑΚΤΙΚΗ ΕΡΓΑΣΙΑ ΦΕΒ. 2025-26

Περιεχόμενα:

Αντικείμενο της εργασίας 2
Παραδοχές 3
Αρχιτεκτονική και επεξήγηση τεχνολογιών υλοποίησης 3
1) MinIO 4
2) Flask API 4
3) RabbitMQ 6
4) Docker 8
Δεδομένα 10
Συμπεράσματα 10

Αντικείμενο της εργασίας

Αντικείμενο της παρούσας εργασίας είναι η δημιουργία ενός συστήματος το οποίο ενσωματώνει την Λάμδα Αρχιτεκτονική. Σκοπός είναι η εξοικείωση με τις τεχνολογίες νέφους, από τις οποίες χρησιμοποιούνται οι RabbitMQ και MinIO. Με την χρήση των εργαλείων αυτών προσομοιώνεται μια όσο το δυνατόν πιο ρεαλιστική ροή εργασίας, πάνω στην λογική της αρχιτεκτονικής νέφους. Το σενάριο το οποίο υλοποιείται, αφορά την λήψη δεδομένων από το API του Open Meteo, επεξεργασία τους και αποθήκευση τους στο MinIo. Επιπλέον, εκπαιδεύεται ένα μοντέλο μηχανικής μάθησης Γραμμικής Παλινδρόμησης, από το οποίο εξάγεται το μέσο σφάλμα καθώς και η πρόβλεψη της θερμοκρασίας της επόμενης ώρας. Τα δεδομένα και η θερμοκρασία αφορούν εξ΄ολοκλήρου την πόλη της Πάτρας. Στη συνέχεια, εφόσον η πρόβλεψη του μοντέλου, η οποία αφορά τη θερμοκρασία των επόμενων 3 ωρών, υπερβεί το όριο που έχουμε θέσει, το οποίο είναι 2*RMSE, στέλνεται Alert Message μέσω του εργαλείου RabbitMQ. Τέλος, ολόκληρο το σύστημα υλοποιείται με Container λογική με το εργαλείο Docker. Οι κλήσεις πραγματοποιούνται με API χρησιμοποιώντας την βιβλιοθήκη Flask.

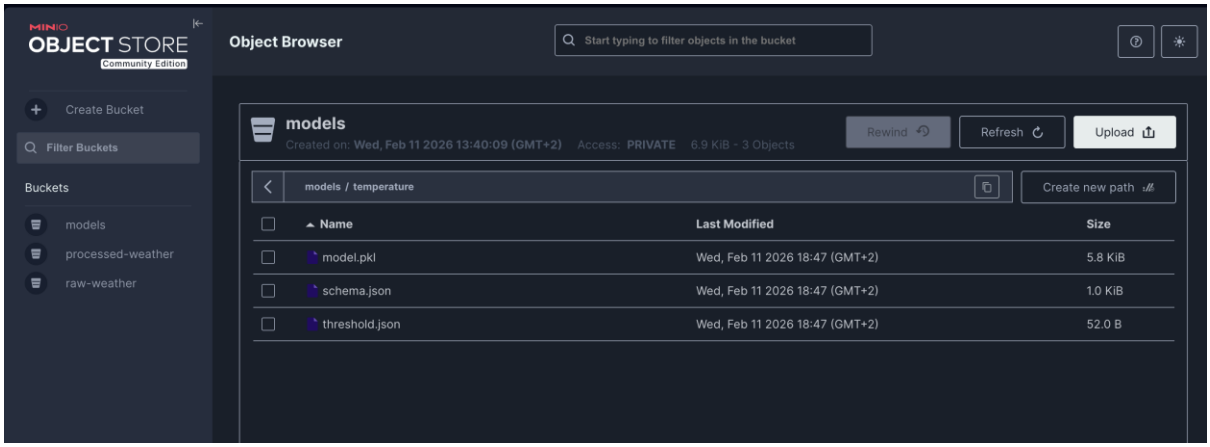
Παραδοχές

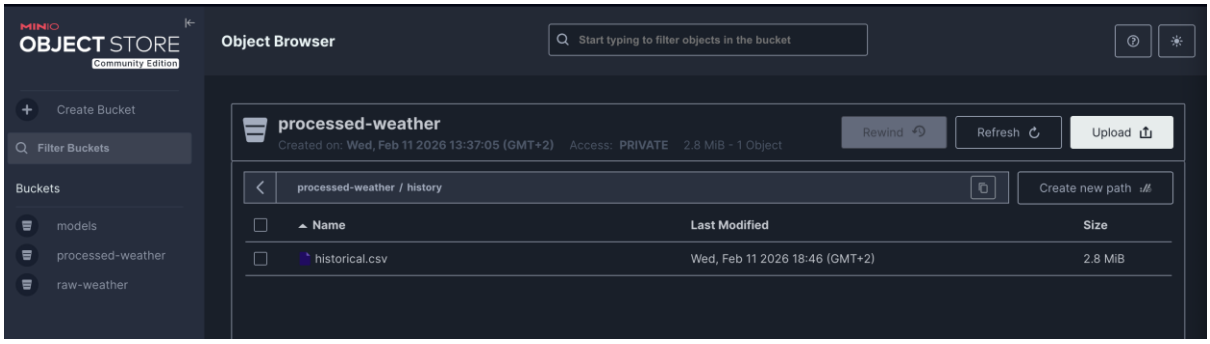
Το σύστημα, όπως προαναφέρθηκε, υλοποιεί προσομοίωση Cloud/Λάμδα αρχιτεκτονικής και σκοπός είναι η ανάδειξη κυρίως των τεχνολογιών που αφορούν την αρχιτεκτονική αυτή. Ως παραδοχή θα πρέπει να αναφερθεί πως το μοντέλο μηχανικής μάθησης, λειτουργεί σε επίπεδο δοκιμών και οι τιμές δεν ανταποκρίνονται σε μεγάλο βαθμό στην πραγματικότητα. Θελήσαμε να δώσουμε έμφαση κυρίως στην λειτουργία της ουσίας του συστήματος, με απόρροια την σχετικά απλοϊκή λειτουργία των προβλέψεων.

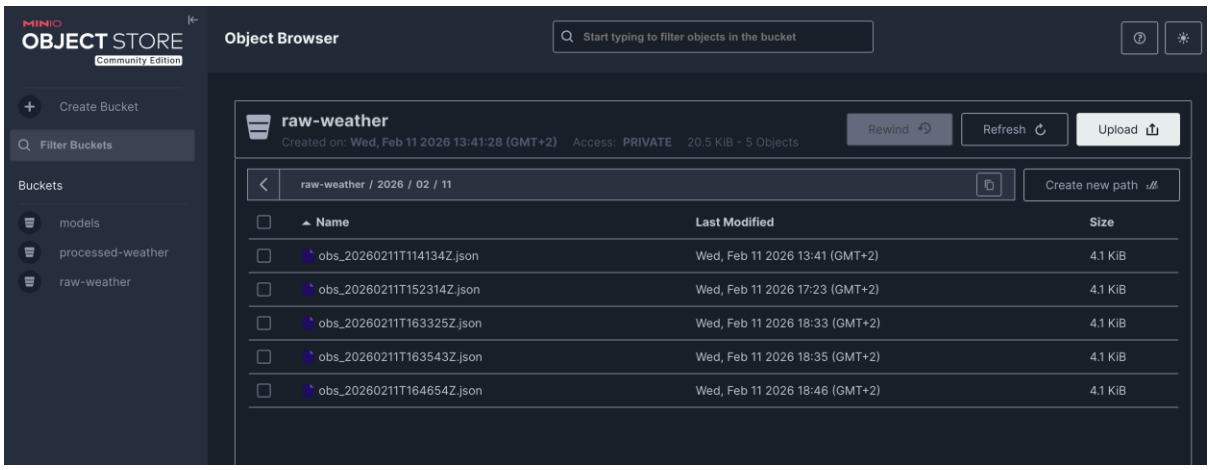
Αρχιτεκτονική και επεξήγηση τεχνολογιών υλοποίησης

Χρησιμοποιούνται οι εξής τεχνολογίες:

1) **MinIO**: Object Storage, στο οποίο αποθηκεύονται raw observation data από το Open Meteo, processed data και model artifacts.
   Δημιουργήθηκαν τα εξής Buckets: models, στο οποίο αποθηκεύεται το μοντέλο, το schema και το threshold, processed-weather στο οποίο αποθηκεύεται το historical.csv, raw-weather στο οποίο αποθηκεύονται τα raw observations.







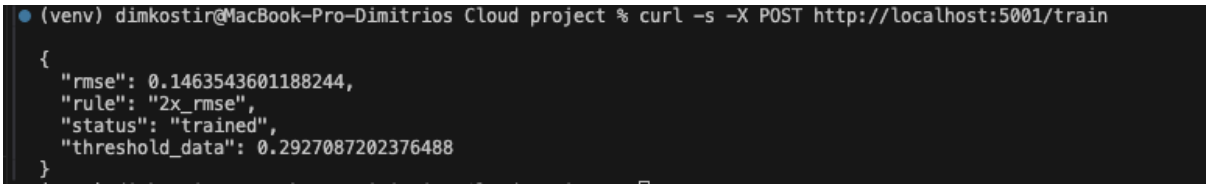
2) **Flask API**: για τη δημιουργία των endpoints /train, /predict, /schema, φόρτωση των ML Artfacts από το MinIO και αποστολή των προβλέψεων

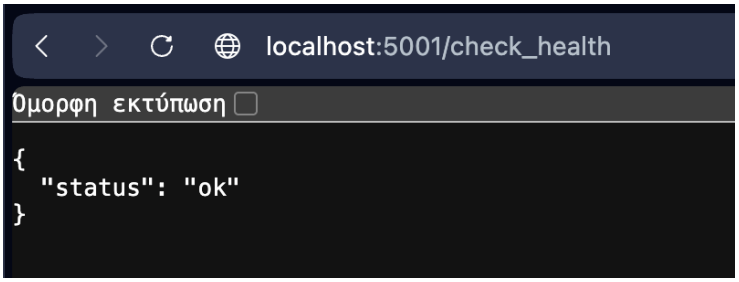
Διαδικασία Train:

- Το endpoint κατεβάζει από το MinIO το αρχείο historical.csv
- Το ανοίγει με Pandas
- Μετατρέπει date σε datetime, φτιάχνει επιπλέον features
- Κάνει καθαρισμό και μετατροπές τιμών
- Δημιουργεί X, y
- y = temperature_2m (target)
- X = όλα τα άλλα columns εκτός από target
- Κάνει preprocessing
- Train/test split
- Train model
- Αποθήκευση model.pkl, threshold.json, schema.json στο MinIO

Διαδικασία Prediction:

- Το endpoint λαμβάνει JSON με το feature subset
- Συμπληρώνει τυχόν ελλιπή πεδία με τα defaults (απλοική λογική ML)
- Δημιουργεί Data Frame με τη σωστή σειρά των στηλών
- Φορτώνει το αποθηκευμένο μοντέλο από το MinIO (model.pkl)
- Εκτελεί την πρόβλεψη

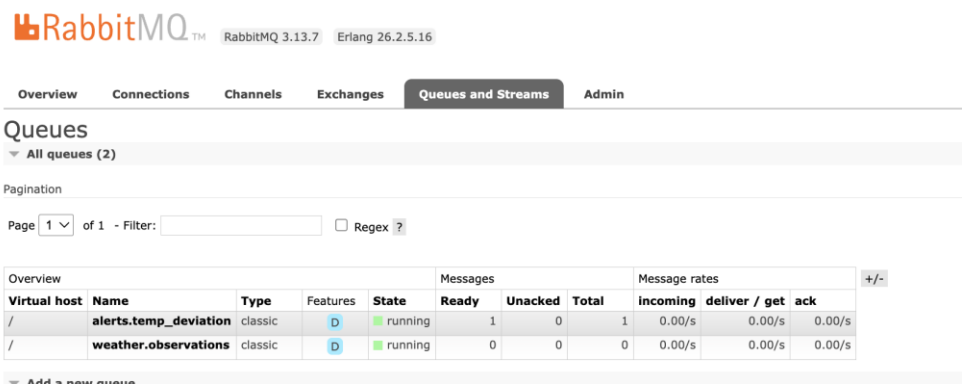


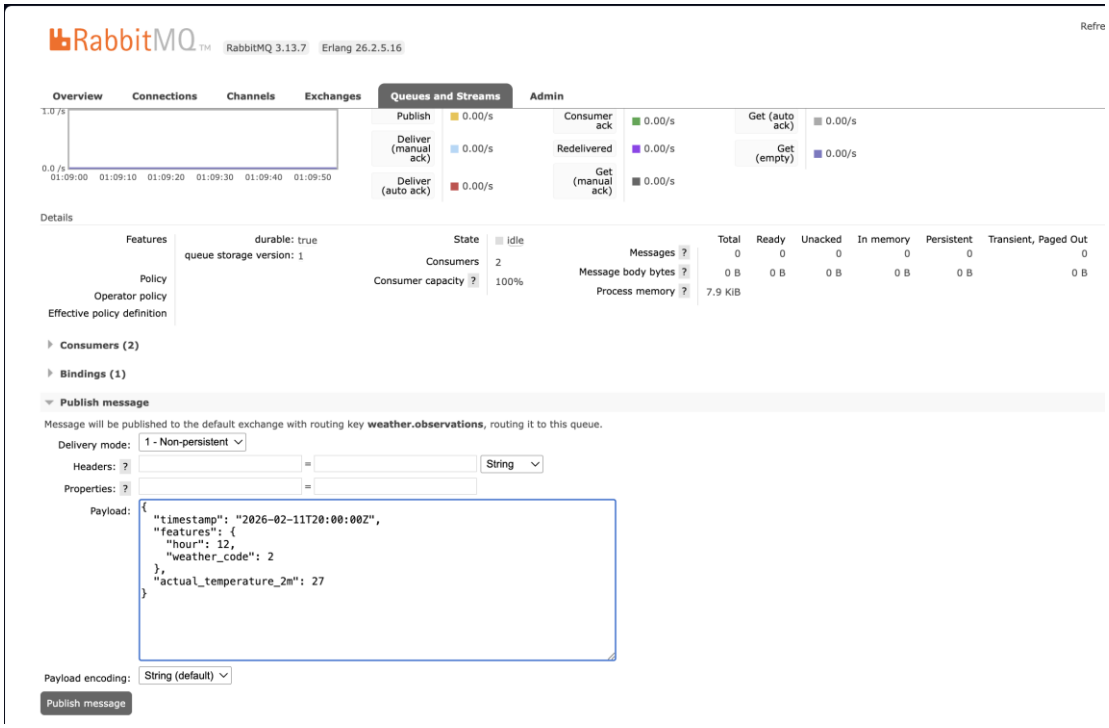






3) **RabbitMQ**: λειτουργεί ως Message Broker για τα observations και τα alerts
   Στις εικόνες φαίνεται η αποστολή observation, η οποία υπερβαίνει το όριο με αποτέλεσμα το Alert Message.



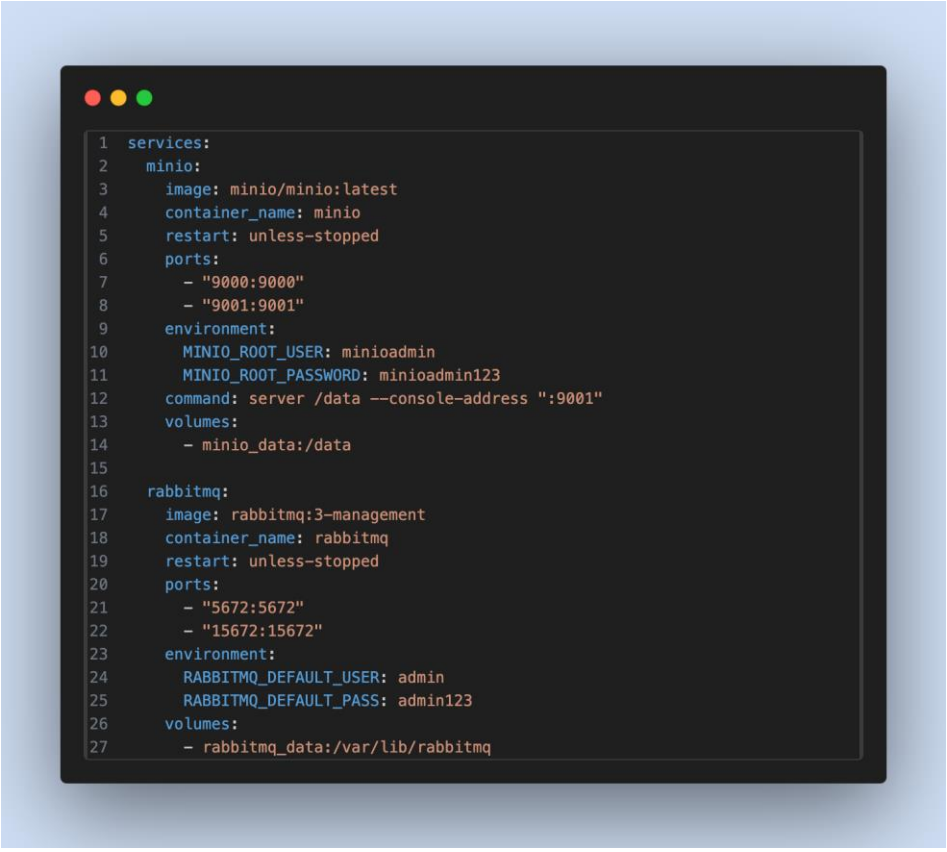




Παρατηρούμε ότι στο Message συμπεριλαμβάνεται η “πραγματική” θερμοκρασία. Είναι πολύ σημαντικό να τονιστεί πως είναι απαραίτητη η υποβολή της, καθώς μόνο

έτσι θα μπορέσει να πραγματοποιηθεί ο υπολογισμός της διαφοράς από την πρόβλεψη.

4) **Docker**: πλήρης δημιουργία services, containers, env variables
   Το σύστημα μπορεί να αναπαραχθεί με την εντολή docker compose up.
   Ενδεικτικά ένα μέρος από το docker-compose.yml



```yaml
services:
  minio:
    image: minio/minio:latest
    container_name: minio
    restart: unless-stopped
    ports:
      - "9000:9000"
      - "9001:9001"
    environment:
      MINIO_ROOT_USER: minioadmin
      MINIO_ROOT_PASSWORD: minioadmin123
    command: server /data --console-address ":9001"
    volumes:
      - minio_data:/data

  rabbitmq:
    image: rabbitmq:3-management
    container_name: rabbitmq
    restart: unless-stopped
    ports:
      - "5672:5672"
      - "15672:15672"
    environment:
      RABBITMQ_DEFAULT_USER: admin
      RABBITMQ_DEFAULT_PASS: admin123
    volumes:
      - rabbitmq_data:/var/lib/rabbitmq
```

Services/APIs/ Functions που υλοποιούνται:

5) **Data_ingestion.py**: Υλοποιεί τη “γέφυρα” προς το MinIO (S3-compatible). Περιέχει helper όπως get_s3_client() που διαβάζει τα MinIO env vars (endpoint/access/secret) και επιστρέφει boto3 client. Όλα τα υπόλοιπα services (trainer, flask, processor) το χρησιμοποιούν για upload/download των CSV data και του ML Model.



```python
def get_s3_client():
    endpoint = os.environ["MINIO_ENDPOINT"]
    access_key = os.environ["MINIO_ACCESS_KEY"]
    secret_key = os.environ["MINIO_SECRET_KEY"]
    return boto3.client(
        "s3",
        endpoint_url=endpoint,
        aws_access_key_id=access_key,
        aws_secret_access_key=secret_key,
        config=Config(signature_version="s3v4"),
        region_name="us-east-1",
    )


def upload_json(bucket: str, key: str, payload: dict):
    s3 = get_s3_client()
    body = json.dumps(payload, ensure_ascii=False).encode("utf-8")
    s3.put_object(
        Bucket=bucket,
        Key=key,
        Body=body,
        ContentType="application/json",
    )


def main():
    bucket = os.environ.get("MINIO_BUCKET_RAW", "raw-weather")

    lat = float(os.environ.get("LATITUDE", "38.2466"))    # Patras default
    lon = float(os.environ.get("LONGITUDE", "21.7346"))

    data = fetch_current_and_past_hours(lat, lon, past_hours=int(os.environ.
get("PAST_HOURS", "24")))

    now = datetime.now(timezone.utc)
    key = now.strftime("%Y/%m/%d") + "/" + now.strftime("obs_%Y%m%d
T%H%M%SZ.json")

    upload_json(bucket=bucket, key=key, payload=data)
    print(f"Uploaded snapshot to s3://{bucket}/{key}")


if __name__ == "__main__":
    main()
```

6) **Client.py**: Ουσιαστικά αποτελεί το training module. Κατεβάζει το historical.csv από το MinIO, χτίζει feature matrix X και target y=temperature_2m, φτιάχνει τα time features (year/month/day/hour/weekday/weekofyear/quarter), καθαρισμό τύπων, split σε train/test και εκπαιδεύει ένα scikit-learn Pipeline, το οποίο χρησιμοποιεί: imputer, scaler, onehot και τέλος LinearRegression. Υπολογίζει RMSE, φτιάχνει το threshold rule (2×RMSE) και δημιουργεί schema (feature_columns + defaults + categorical_features). Τέλος κάνει upload τα artifacts (model.pkl, threshold.json, schema.json) στο MinIO.



```python
def train_and_build_artifacts() -> Tuple[Pipeline, float, Dict[str, Any], Dict[str, Any]]:
    """
    Trains a model pipeline and returns:
      - model pipeline
      - rmse
      - threshold_data
      - schema (feature_columns + defaults)
    """
    df = load_historical()
    X, y = build_training_frame(df)

    # Decide categorical columns (keep it simple)
    categorical_features: List[str] = []
    if "weather_code" in X.columns:
        categorical_features.append("weather_code")
    if "is_day" in X.columns:
        categorical_features.append("is_day")

    numeric_features = [c for c in X.columns if c not in categorical_features]

    numeric_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler()),
        ]
    )

    categorical_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("onehot", OneHotEncoder(handle_unknown="ignore")),
        ]
    )

    preprocessor = ColumnTransformer(
        transformers=[
            ("num", numeric_transformer, numeric_features),
            ("cat", categorical_transformer, categorical_features),
        ],
        remainder="drop",
    )

    model = Pipeline(
        steps=[
            ("preprocessor", preprocessor),
            ("regressor", LinearRegression()),
        ]
    )

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    rmse = float(root_mean_squared_error(y_test, y_pred))

    threshold_data = {
        "rule": "2x_rmse",
        "threshold": 2.0 * rmse,
    }

    schema = {
        "target": TARGET_COL,
        "feature_columns": list(X.columns),
        "categorical_features": categorical_features,
        "defaults": compute_schema_defaults(X_train, categorical_features),
    }

    return model, rmse, threshold_data, schema
```



```python
def save_and_upload_artifacts(model: Pipeline, threshold_data: Dict[str, Any
], schema: Dict[str, Any]) -> None:
    """
    Saves model.pkl, threshold.json, schema.json locally and uploads to MinIO.
    """
    s3 = get_s3_client()

    model_bucket = os.environ["MODEL_BUCKET"]
    model_key = os.environ["MODEL_KEY"]
    thresh_key = os.environ["THRESHOLD_KEY"]
    schema_key = os.environ.get("SCHEMA_KEY", "temperature/schema.json")

    joblib.dump(model, "model.pkl")
    with open("threshold.json", "w") as f:
        json.dump(threshold_data, f)
    with open("schema.json", "w") as f:
        json.dump(schema, f)

    s3.upload_file("model.pkl", model_bucket, model_key)
    s3.upload_file("threshold.json", model_bucket, thresh_key)
    s3.upload_file("schema.json", model_bucket, schema_key)


def run_training_pipeline() -> Tuple[float, Dict[str, Any]]:
    """
    Trains model + uploads artifacts. Returns rmse and threshold_data.
    """
    model, rmse, threshold_data, schema = train_and_build_artifacts()
    save_and_upload_artifacts(model, threshold_data, schema)
    return rmse, threshold_data
```

7) **Main.py:** Αποτελεί το serving layer. Παρέχει ουσιαστικά τα endpoints /train (τρέχει training pipeline και ανεβάζει artifacts), /schema (επιστρέφει τι περιμένει το μοντέλο και ποια defaults θα χρησιμοποιήσει), και /predict (φορτώνει artifacts από MinIO, φτιάχνει μία γραμμή DataFrame με σωστή σειρά features, συμπληρώνει όσα λείπουν με defaults, και επιστρέφει prediction + threshold/rule). Είναι ο τρόπος για την λειτουργία των API που χρησιμοποιεί και βλέπει ο χρήστης με curl και ο Rabbit consumer.



```python
@app.get("/check_health")
def check_health():
    return jsonify({"status": "ok"}), 200


@app.post("/train")
def train():
    try:
        rmse, threshold_data = run_training_pipeline()
    except Exception as e:
        return jsonify({"error": "Training failed", "detail": str(e)}), 500

    return jsonify(
        {
            "status": "trained",
            "rmse": float(rmse),
            "threshold_data": float(threshold_data["threshold"]),
            "rule": threshold_data.get("rule", "2x_rmse"),
        }
    ), 200


@app.post("/predict")
def predict():
    payload = request.get_json(silent=True) or {}
    features = payload.get("features", None)

    if not isinstance(features, dict):
        return jsonify({"error":
"Payload must be JSON with a 'features' object"}), 400

    try:
        model, threshold_data, schema = load_artifacts()
    except Exception as e:
        return (
            jsonify(
                {
                    "error":
"Model artifacts not found. Run POST /train first.",
                    "detail": str(e),
                }
            ),
            409,
        )

    X_row = build_row_from_payload(schema, features)
    pred = float(model.predict(X_row)[0])

    return jsonify(
        {
            "predicted_temperature_2m": pred,
            "horizon_hours": 3,
            "threshold": float(threshold_data["threshold"]),
            "rule": threshold_data.get("rule", "2x_rmse"),
        }
    ), 200
```

8) **Rabbit.py**: Αποτελεί την υλοποίηση του Message System. Συνδέεται στο RabbitMQ, καταναλώνει ουσιαστικά τα messages από το observation queue, καλεί το /predict endpoint και υπολογίζει την απόλυτη διαφορά |actual_temperature-predicted_temperature|. Αν η διαφορά αυτή είναι μεγαλύτερη από το threshold (2*RMSE), τότε στέλνει Alert Message στο Deviations Queue.



```python
def connect_with_retry(max_tries=30, sleep_seconds=2):
    creds = pika.PlainCredentials(RABBIT_USER, RABBIT_PASS)
    params = pika.ConnectionParameters(
        host=RABBIT_HOST,
        port=RABBIT_PORT,
        credentials=creds,
        heartbeat=30,
        blocked_connection_timeout=30,
    )

    for i in range(max_tries):
        try:
            print(f"Connecting to RabbitMQ... ({i+1}/{max_tries})")
            return pika.BlockingConnection(params)
        except Exception as e:
            print(f"RabbitMQ not ready: {e}")
            time.sleep(sleep_seconds)

    raise RuntimeError("Could not connect to RabbitMQ after retries")


def main():
    connection = connect_with_retry()
    channel = connection.channel()

    channel.queue_declare(queue=OBS_QUEUE, durable=True)
    channel.queue_declare(queue=ALERT_QUEUE, durable=True)
    channel.basic_qos(prefetch_count=1)

    def on_message(ch, method, properties, body):
        try:
            msg = json.loads(body.decode("utf-8"))
        except Exception:
            ch.basic_ack(delivery_tag=method.delivery_tag)
            return

        features = msg.get("features")
        if not isinstance(features, dict):
            ch.basic_ack(delivery_tag=method.delivery_tag)
            return

        # Call API /predict
        try:
            resp = requests.post(PREDICT_URL, json={"features": features},
timeout=15)
        except Exception:
            ch.basic_nack(delivery_tag=method.delivery_tag, requeue=True)
            return

        if resp.status_code != 200:
            if 500 <= resp.status_code <= 599:
                ch.basic_nack(delivery_tag=method.delivery_tag, requeue=True)
            else:
                ch.basic_ack(delivery_tag=method.delivery_tag)
            return

        result = resp.json()
        predicted = float(result["predicted_temperature_2m"])
        threshold = float(result["threshold"])

        actual = msg.get("actual_temperature_2m")

        if actual is not None:
            error = abs(float(actual) - predicted)

            if error > threshold:
                alert = {
                    "status": "ALARM",
                    "predicted_temperature_2m": predicted,
                    "actual_temperature_2m": float(actual),
                    "error": error,
                    "threshold": threshold,
                    "rule": result.get("rule", "2x_rmse"),
                    "timestamp": msg.get("timestamp"),
                }

                channel.basic_publish(
                    exchange="",
                    routing_key=ALERT_QUEUE,
                    body=json.dumps(alert).encode("utf-8"),
                    properties=pika.BasicProperties(delivery_mode=2),
                )

        ch.basic_ack(delivery_tag=method.delivery_tag)

    channel.basic_consume(queue=OBS_QUEUE, on_message_callback=on_message)
    print(f"Consumer listening on queue: {OBS_QUEUE}")
    channel.start_consuming()


if __name__ == "__main__":
    main()
```

9) **Processor.py**: Μετασχηματίζει τα raw observation αρχεία τα οποία έχουν αποθηκευτεί στο MinIO, και τα αποθηκεύει εκ νέου σε bucket processed-weather. Επιπλέον, ενημερώνει το historical.csv, το οποίο χρησιμοποιεί ως dataset για το training.



```python
def raw_json_to_rows(raw: dict) -> list[dict]:
    rows = []
    hourly = raw.get("hourly", {})
    times = hourly.get("time", [])
    if not times:
        return rows

    # Take all hourly variables except "time"
    hourly_vars = [k for k in hourly.keys() if k != "time"]

    for i, t in enumerate(times):
        dt = pd.to_datetime(t)
        # Keep every 3 hours
        if dt.hour % 3 != 0:
            continue

        row = {"date": dt.isoformat()}
        for var in hourly_vars:
            vals = hourly.get(var, [])
            if i < len(vals):
                row[var] = vals[i]
        rows.append(row)

    return rows


def build_new_df_from_raw_bucket(s3, raw_bucket: str, raw_prefix: str) -> pd.DataFrame:
    keys = list_raw_keys(s3, raw_bucket, raw_prefix)

    all_rows = []
    for key in keys:
        if not key.endswith(".json"):
            continue

        local_path = "/tmp/raw.json"
        s3.download_file(raw_bucket, key, local_path)

        with open(local_path, "r") as f:
            raw = json.load(f)

        all_rows.extend(raw_json_to_rows(raw))

    if not all_rows:
        return pd.DataFrame()

    df_new = pd.DataFrame(all_rows)
    df_new["date"] = pd.to_datetime(df_new["date"])
    df_new = df_new.sort_values("date").drop_duplicates(subset=["date"], keep="last")
    return df_new


def upsert_historical_csv(s3, processed_bucket: str, historical_key: str, df_new: pd.DataFrame) -> int:
    if df_new.empty:
        return 0

    local_hist = "/tmp/historical.csv"

    # Try download existing historical.csv
    try:
        s3.download_file(processed_bucket, historical_key, local_hist)
        df_old = pd.read_csv(local_hist)
        df_old["date"] = pd.to_datetime(df_old["date"])
        df_all = pd.concat([df_old, df_new], ignore_index=True)
    except Exception:
        # If not exists, create new
        df_all = df_new.copy()

    df_all["date"] = pd.to_datetime(df_all["date"], utc=True).dt.tz_convert(None)

    df_all = df_all.sort_values("date").drop_duplicates(subset=["date"], keep="last")
    df_all.to_csv(local_hist, index=False)

    s3.upload_file(local_hist, processed_bucket, historical_key)
    return len(df_new)
```

Δεδομένα

1.Συλλογή δεδομένων

Το σύστημα αντλεί δεδομένα από το Open Meteo, χρησιμοποιώντας το API του. Τα δεδομένα περιλαμβάνουν τιμές όπως θερμοκρασία αέρα, σχετική υγρασία, σημείο δροσιάς, αισθητή θερμοκρασία κ.ά. Τα δεδομένα αυτά αποθηκεύονται σε bucket στο MinIO, ως παρατηρήσεις τόσο ημερήσια όσο και στιγμιαία. Δηλαδή κάθε observation περιέχει τόσο current snapshots, όσο και ημερήσια στοιχεία.

2.Επεξεργασία δεδομένων

Πραγματοποιείται επεξεργασία των δεδομένων, ουσιαστικά προετοιμάζοντας τα για την εφαρμογή στον αλγόριθμο μηχανικής μάθησης. Υφίσταται μετατροπή των string τιμών σε numeric, scaling, καθώς και καθαρισμός τυχόν ελλιπών τιμών.

Δημιουργούνται επίσης χρονικά χαρακτηριστικά, τα οποία αντλούνται από το date:

- Year
- Month
- Day
- Hour
- Weekday
- Weekofyear
- Quarter

3.Training Dataset

Στη συνέχεια δημιουργείται training dataset, με τιμή στόχο την temperature_2m, και features όλα τα υπόλοιπα πλην του στόχου. Καθίσταται σαφές πως η μεταβλητή στόχος δεν περιλαμβάνεται ποτέ στα features, ώστε να αποφευχθεί data leakage.

4.Εκπαίδευση μοντέλου

Χρήση Linear Regression. Υλοποιείται μέσα από Pipeline, για σωστή και ασφαλή εφαρμογή των βημάτων.

Υπολογίζεται το μέσο σφάλμα του μοντέλου (RMSE) και στη συνέχεια υπολογίζεται το όριο (threshold), το οποίο όπως προαναφέρθηκε είναι 2*RMSE.

5.Συμπλήρωση default τιμών

Όπως αναφέρθηκε και στην ενότητα “Παραδοχές”, στόχος του συστήματος είναι η υλοποίηση της Cloud λογικής και αρχιτεκτονικής. Εξαιτίας της φύσης των δεδομένων καιρού και τις δυσκολίας αποτελεσματικής και επιστημονικά ορθής πρόβλεψης, η υλοποίηση της

ML λογικής είναι σε μορφή τέτοια έτσι ώστε το σύστημα να είναι λειτουργικό, όσο το δυνατόν πιο αποτελεσματικά. Για το λόγο αυτό, χρησιμοποιούνται default τιμές, στην περίπτωση που το Payload αφορά ελάχιστα χαρακτηριστικά. Έτσι εξασφαλίζεται η πρόβλεψη σταθερών και φυσιολογικών τιμών, οι οποίες μας επιτρέπουν την ορθή λειτουργία του συστήματος, σε Cloud-based λογική.

Συμπεράσματα

Το σύστημα υλοποιεί αποτελεσματικά την Λάμδα Αρχιτεκτονική, εφαρμόζοντας επεξεργασία πραγματικών δεδομένων χρησιμοποιώντας τεχνικές Μηχανικής Μάθησης.

Αναμφίβολα, το σύστημα είναι επεκτάσιμο, ενώ η διαχείριση δεδομένων διασφαλίζει σταθερή, παραγωγική λειτουργία. Τέλος, τα δεδομένα, επεξεργάζονται με τρόπο τέτοιο ο οποίος θα αποδίδει αποτελέσματα ικανά προς επεξεργασία και αποθήκευση.